
Xarxes neuronals recurrents

PID_00290593

Toni Lozano Bagén

Temps mínim de dedicació recomanat: 3 hores



Universitat
Oberta
de Catalunya

Toni Lozano Bagén

L'encàrrec i la creació d'aquest recurs d'aprenentatge UOC han estat coordinats pel professor: Jordi Casas Roma

Com citar aquest recurs d'aprenentatge amb l'estil Harvard:

Lozano Bagén, T. (2023) *Xarxes neuronals recurrents*. [Recurs d'aprenentatge textual]. 1a edició. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edició: febrer 2024

© d'aquesta edició, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoria: Toni Lozano Bagén

Producció: FUOC

Tots els drets reservats

Cap part d'aquesta publicació, incloent-hi el disseny general i la coberta, no pot ser copiada, reproduïda, emmagatzemada o transmesa de cap manera ni per cap mitjà, ja sigui elèctric, químic, mecànic, òptic, enregistrament, fotocòpia, o qualsevol altre, sense la prèvia autorització escrita dels titulars dels drets.

Índex

Introducció	4
Objectius	5
1. Fonaments de les xarxes recurrents	6
1.1. Concepte de recurrència	6
1.2. Tipus de xarxes neuronals recurrents	10
1.3. Entrenament d'una xarxa neuronal recurrent	12
2. Tipologia de cel·les recurrents	14
2.1. <i>Long Short Term Memory</i> (LSTM)	14
2.1.1. Estructura de les cel·les	14
2.1.2. Consideracions sobre les cel·les LSTM	17
2.2. <i>Gated Recurrent Unit</i> (GRU)	18
2.2.1. Estructura de les cel·les	18
2.2.2. Consideracions sobre les cel·les GRU	20
3. Arquitectures de xarxes recurrents	21
3.1. Xarxes neuronals recurrents bidireccionals	21
3.2. Xarxes neuronals recurrents profundes	22
3.3. Arquitectura codificador-descodificador	24
3.4. Mecanisme d'atenció	25
4. Consells pràctics i exemples	28
4.1. Consells pràctics en l'ús d'RNN	28
4.2. Exemples	31
Resum	36
Glossari	37
Bibliografia	38

Introducció

Quan un conjunt de dades té una estructura intrínseca seqüencial, com els textos o les sèries temporals, existeix una correlació espacial entre els elements de cada registre en el conjunt de dades. En aquests casos podem definir les connexions en una xarxa neuronal de manera que l'entrenament de la xarxa aprofiti al màxim l'estructura seqüencial de les dades. El resultat són les xarxes neuronals recurrents.

Una xarxa neuronal recurrent es pot construir de diverses formes. En aquest mòdul didàctic veurem les formes més habituals de construir xarxes neuronals recurrents i alguns casos d'ús.

Concretament, en el primer capítol veurem el concepte fonamental de les xarxes neuronals recurrents, la recurrència, juntament amb els tipus de xarxes neuronals recurrents que es poden considerar segons les característiques del problema que es vol resoldre. Veurem també com es pot utilitzar l'algorisme de retropropagació per entrenar una xarxa neuronal recurrent.

En el segon capítol analitzarem detalladament l'estructura interna dels dos tipus de cel·les més comunament utilitzades en una xarxa neuronal recurrent: les cel·les LSTM (*Long Short Term Memory*) i les cel·les GRU (*Gated Recurrent Memory*).

El tercer capítol està dedicat a considerar les diferents formes en les quals es poden connectar les cel·les d'una xarxa neuronal recurrent per aconseguir models amb major capacitat predictiva. S'introduirà també en aquest capítol el concepte d'atenció, que és la base de l'arquitectura de transformadors que veurem en futurs mòduls didàctics.

Finalment, en el quart capítol es veurà com a exemple l'ús d'una xarxa neuronal recurrent per la predicció d'una sèrie temporal generada sintèticament.

Objectius

En els materials didàctics d'aquest mòdul trobarem les eines indispensables per assimilar els objectius següents:

1. Entendre el cas d'ús de les xarxes neuronals recurrents.
2. Comprendre els principals tipus d'operacions que realitza una xarxa neuronal recurrent.
3. Conèixer les diferents arquitectures que es poden utilitzar amb xarxes neuronals recurrents.

1. Fonaments de les xarxes recurrents

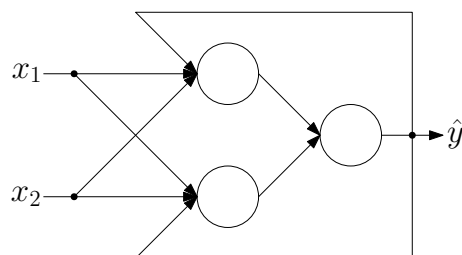
En aquest capítol introduïrem els principis i els conceptes fonamentals de les xarxes neuronals recurrents. En concret, iniciarem aquest capítol revisant el concepte de «recurrència» que s'aplica en aquesta mena de xarxes i, a continuació, veurem els principals tipus de xarxes neuronals recurrents. Finalitzarem el capítol discutint el mètode d'entrenament en el cas de les xarxes recurrents.

1.1. Concepte de recurrència

Fins ara, en totes les xarxes neuronals que hem vist, la informació flueix sempre en una única direcció: des de les neurones d'entrada cap a les neurones de sortida. Aquest tipus de xarxes neuronals es coneixen en anglès amb el terme *feedforward*, però, en general, en una xarxa neuronal pot haver-hi connexions que vagin des d'una capa posterior cap a una capa anterior.

A la figura 1 podem veure un exemple en el qual la capa de neurones ocultes està connectada a les dades d'entrada i, al seu torn, a la sortida de les neurones de la pròpia xarxa neuronal. D'aquesta forma, quan la xarxa processa un registre concret està utilitzant, alhora, informació del propi registre i informació que va generar la xarxa amb el registre anterior. Això crea un estat intern a la xarxa que li permet variar la resposta en un cert registre d'entrada depenent dels registres que ha processat la xarxa anteriorment. Com veurem més endavant, en molts casos a l'estat de la xarxa també se'l denomina **memòria** de la xarxa.

Figura 1. Exemple de xarxa neuronal recurrent



Aquest tipus de connexions poden ser molt útils per tractar seqüències de dades, com poden ser les sèries temporals o els textos. Per exemple, si volem fer anàlisi de sentiment d'una frase, podem introduir les paraules de la frase una a una a la xarxa neuronal, de manera que en cada pas de la xarxa l'estat va canviant fins que introduïm l'última paraula de la frase, moment en el qual

observem la sortida de la xarxa per determinar si la frase té sentiment positiu o negatiu.

Observem que, en principi, res no impedeix tractar seqüències de dades amb una xarxa completament connectada com les que hem vist en mòduls anteriors. Per a això hauríem de fixar *a priori* la longitud de les seqüències a tractar i aniríem introduint el primer valor de la seqüència a la primera neurona, el segon valor a la segona neurona, etc. Un dels majors problemes amb aquesta solució apareix quan existeixen seqüències que tenen els mateixos valors en diferent ordre però així i tot tenen el mateix significat. Un exemple d'això serien les frases «La pel·lícula que vaig veure ahir no em va agradar gaire» i «No em va agradar gaire la pel·lícula que vaig veure ahir». En una xarxa completament connectada, les neurones tenen un ordre fix, per la qual cosa reaccionen a les dades d'entrada en un ordre concret. Això vol dir que, en aquesta mena de problemes, on l'ordre de les paraules no és el més important, hauríem d'entrenar la xarxa neuronal completament connectada utilitzant totes les variacions possibles de cada frase, la qual cosa, en la majoria dels casos, és inviable.

En els casos on l'ordre de les paraules sí que influeix en el significat de la frase també és més interessant utilitzar xarxes neuronals recurrents en lloc de xarxes neuronals completament connectades. La raó és que les neurones en una xarxa neuronal completament connectada són «independents entre si», per la qual cosa és difícil que una xarxa neuronal completament connectada sigui capaç d'aprendre que el significat depèn de l'ordre de les paraules. En una xarxa neuronal recurrent, en canvi, les paraules es processen en ordre, per la qual cosa estem forçant la xarxa al fet que consideri l'ordre com una font d'informació important.

En general, les connexions recurrents en una xarxa neuronal poden ser de molts tipus. Per aquesta raó, i per poder fer un estudi unificat de les xarxes neuronals recurrents, es defineix el concepte de «cel·la» (en anglès, *cell*). Una **cel·la** és simplement una operació amb dues entrades i dues sortides que es va component amb si mateixa a mesura que s'aplica l'operació als diferents valors de la seqüència. Els punts d'entrada de la cel·la corresponen, d'una banda, als valors de la seqüència per a cada registre i, d'altra banda, a l'estat de la xarxa neuronal en el pas anterior. Els punts de sortida solen ser l'estat de la xarxa en el pas concret en el qual s'executa la cel·la i la resposta de la xarxa per aquest pas.

Com veurem més endavant, en alguns casos les entrades i les sortides d'una cel·la poden bifurcar-se o concatenar-se per simular tenir més o menys de dues entrades i dues sortides, però el concepte fonamental de cel·la continua sent el mateix.

Formalment, suposem que denotem el pas concret d'una seqüència amb la lletra t . Si els valors de la seqüència són vectors de dimensió m tenim que, en el temps t , podem denotar el vector d'entrada a la xarxa com $x_t \in \mathbb{R}^m$. De la mateixa forma, la sortida en cada pas de la recurrència pot ser un vector de dimensió l , per la qual cosa tindrem $h_t \in \mathbb{R}^l$. Finalment, hem de fixar un valor n per la dimensió del vector que representa l'estat de la xarxa, de manera que tindrem $s_t \in \mathbb{R}^n$. Amb aquestes consideracions, una cel·la és una operació f com la següent:

$$\begin{aligned} f : \quad \mathbb{R}^{m+n} &\rightarrow \mathbb{R}^{n+l} \\ (x_t, s_{t-1}) &\mapsto (h_t, s_t) \end{aligned} \quad (1)$$

L'operació que calcula la cel·la, f , se sol representar internament amb neurones i connexions no recurrents entre les neurones. A la figura 2 podem observar una cel·la genèrica, on només mostrem les entrades i les sortides, mentre que a la figura 3 podem visualitzar la xarxa de la figura 1 representada amb el format de cel·la.

Figura 2. Diagrama d'una cel·la, sense especificar les operacions internes

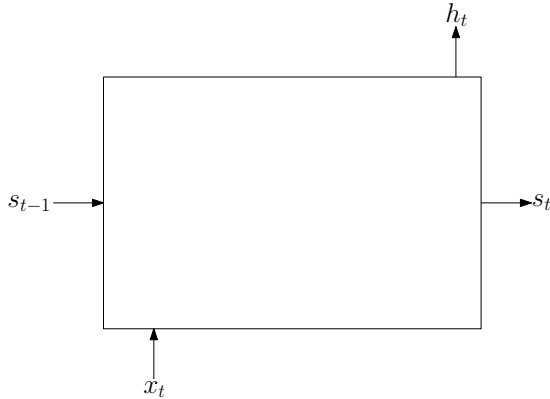
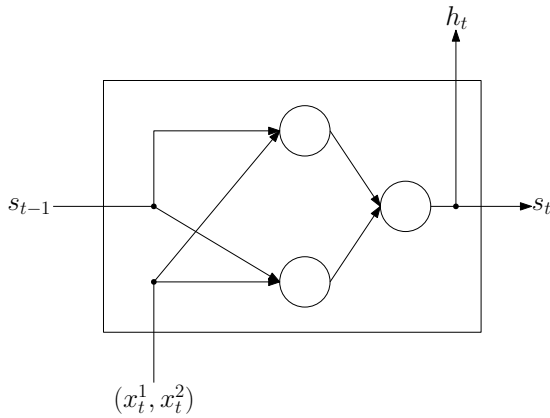


Figura 3. Representació en format cel·la de la xarxa recurrent de la figura 1

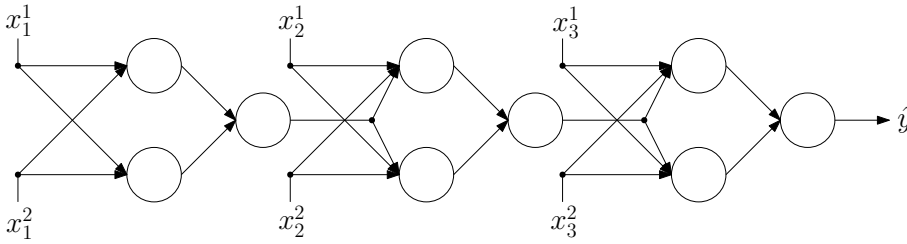


A l'exemple concret de la figura 3, cada connexió representa un únic valor escalar, però, en general, les connexions internes d'una cel·la recurrent poden treballar amb vectors, de manera que les sortides de la cel·la estan formades

també per vectors, com es mostra a l'equació 1. En aquest cas, normalment s'afegeixen una o diverses capes completament connectades a la sortida de l'última cel·la recurrent per adaptar la grandària de la sortida al conjunt de dades amb el qual treballem.

Si considerem una xarxa neuronal recurrent en la seva màxima generalitat, la sortida de la xarxa en un moment donat pot dependre de dades tan antigues com llarga sigui la seqüència. A la pràctica, per fer més eficient l'ús de les xarxes neuronals recurrents, es tria un valor k que fixa la longitud de les seqüències que tractarà la xarxa. Si les seqüències a tractar tenen longitud menor que k s'afegeixen valors especials de farciment (al principi o al final de la seqüència). En canvi, si les seqüències són més llargues del valor triat es tallen i es descarta part de la seqüència (s'ha de triar també si descartar el principi o el final de la seqüència). Una vegada fixat el valor de k podem «desenrotllar» la xarxa per convertir la xarxa neuronal recurrent en una xarxa neuronal *feedforward*, com podem veure a la figura 4.

Figura 4. Xarxa neuronal recurrent desenrotllada amb $k = 3$



És important notar que els paràmetres interns de les cel·les són compartits en tots els passos, per la qual cosa els valors dels paràmetres són iguals en cada pas. La diferència entre un pas i el següent estarà únicament en les entrades a la cel·la que, com hem dit, es corresponen amb l'estat de la xarxa i les dades d'entrada. Concretament, els paràmetres interns de la xarxa fixen la funció $f : \mathbb{R}^{m+n} \rightarrow \mathbb{R}^{n+l}$, de manera que, si fixem el valor inicial per s_0 , podem escriure la recurrència com:

$$(h_1, s_1) = f(x_1, s_0)$$

$$(h_2, s_2) = f(x_2, s_1)$$

$$\vdots$$

$$(h_t, s_t) = f(x_t, s_{t-1})$$

1.2. Tipus de xarxes neuronals recurrents

Com hem comentat, les xarxes neuronals recurrents són especialment útils per tractar amb seqüències de dades, però així i tot podem distingir entre diferents tipus. Depenent del problema en el qual estiguem treballant, pot interessar-nos considerar únicament algunes de les entrades o de les sortides de les cel·les de la xarxa, la qual cosa dona lloc als tipus de xarxes recurrents següents:

- Si les seqüències que tractem tenen diverses dades d'entrada però només ens interessa un únic valor que observem al final de la recurrència, obtenim un diagrama com el de la figura 5. Un exemple d'aquest cas d'ús seria fer una anàlisi de sentiment sobre textos.
- Si treballem amb seqüències que contenen un únic valor però ens interessa generar una sortida amb diversos passos, podem introduir un valor nul a partir del segon pas de la recurrència i anar observant la sortida de la xarxa, com apareix a la figura 6. Aquest tipus de xarxa recurrent es pot utilitzar per exemple per obtenir descripcions donada una única imatge d'entrada.
- En el cas en què tractem seqüències amb diversos valors i estiguem interessats en els valors que genera la xarxa neuronal una vegada tractada la seqüència d'entrada, podem procedir com en el cas anterior introduint valors nuls una vegada processada la seqüència i observant la sortida de la xarxa, com podem veure a la figura 7. Si treballem amb sèries temporals, podem utilitzar aquest tipus de xarxa recurrent per introduir valors passats a la xarxa i obtenir la predicció de diversos valors a futur.
- És possible que estiguem interessats a treballar amb seqüències que tenen diversos valors i necessitem obtenir una resposta sincronitzada per cadascun dels valors d'entrada, en aquest cas tindríem un diagrama com el de la figura 8. Un cas d'ús d'aquesta mena de xarxa es dona, per exemple, quan es treballa amb un vídeo fotograma a fotograma i es desitgen classificar objectes en cada fotograma.

Figura 5. Xarxa neuronal recurrent amb diverses dades d'entrada i una única sortida

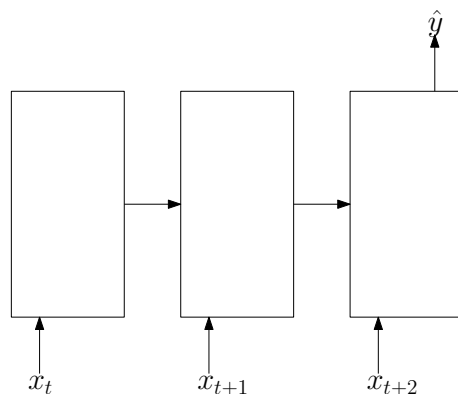


Figura 6. Xarxa neuronal recurrent amb una única dada d'entrada i diverses dades de sortida

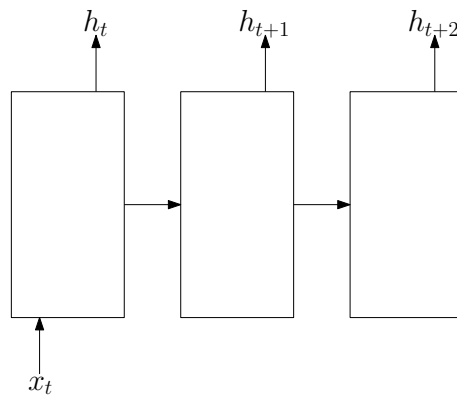


Figura 7. Xarxa neuronal recurrent amb diverses dades d'entrada i diverses dades de sortida

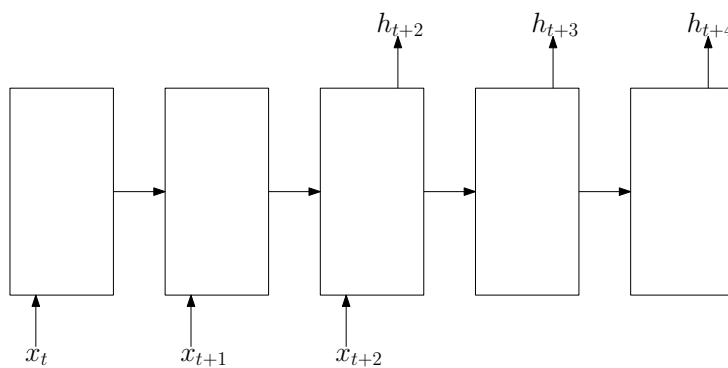
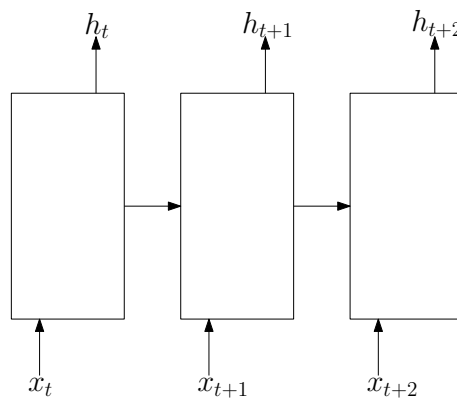


Figura 8. Xarxa neuronal recurrent amb diverses dades d'entrada i diverses dades de sortida sincronitzades



És important observar que totes les cel·les tenen sempre dues entrades i dues sortides. En cadascuna de les figures 5, 6, 7 i 8, si no apareix alguna de les sortides és perquè aquesta sortida en concret s'ignora, mentre que si no apareix alguna de les entrades és perquè en aquests punts s'introdueix un valor concret que actua com a valor nul.

1.3. Entrenament d'una xarxa neuronal recurrent

En el mòdul sobre xarxes neuronals completament connectades hem vist l'algorisme de retropropagació (*backpropagation*), que permet calcular de manera eficient el gradient d'una funció de cost per a una xarxa neuronal *feedforward*. En el cas d'una xarxa neuronal recurrent no es pot aplicar directament l'algorisme de retropropagació a causa de les connexions entre neurones de capes posteriors amb neurones de capes anteriors.

En general, hem vist que entrenar una xarxa neuronal és trobar uns valors per als paràmetres de la xarxa que minimitzin una determinada funció de cost, i això és igual en el cas de les xarxes recurrents. Per aquesta raó, es poden entrenar les xarxes neuronals recurrents amb qualsevol algorisme d'optimització, com per exemple els algorismes genètics. Així i tot, donat l'alt rendiment de l'algorisme de retropropagació per altres tipus de xarxes neuronals, és lògic pensar que una adaptació del mateix algorisme pot resultar útil també en el cas de les xarxes recurrents.

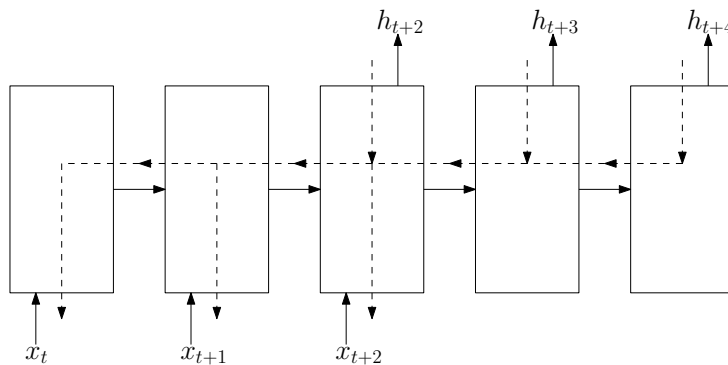
Aquesta adaptació es coneix amb el nom de retropropagació en el temps (en anglès, *backpropagation through time*) i no és més que desenrotllar la xarxa recurrent per convertir-la en una xarxa *feedforward* i aplicar l'algorisme de retropropagació. Cal tenir en compte, això sí, el tipus de xarxa que estiguem utilitzant ja que, segons hem vist a la secció anterior, a vegades ens pot interessar treballar amb la sortida de la xarxa en diferents passos, la qual cosa afecta directament la funció de cost. Recordem que en una xarxa recurrent tots els paràmetres interns de la cel·la són compartits en tots els passos, per la qual cosa el gradient de la funció de cost pot afectar un mateix paràmetre de la xarxa repetides vegades, depenent d'en quins passos s'estiguin considerant les sortides de la xarxa per calcular la funció de cost.

Si ens fixem en la figura 4, podem intuir que el mètode de retropropagació calcularà l'error en cadascuna de les sortides que estiguem considerant i el propagarà cap endarrere en cadascun dels passos, tal com es mostra a la figura 9. Això vol dir que, si considerem la sortida de la xarxa a cada pas, cada paràmetre de la xarxa neuronal contribuirà al gradient de la funció de cost una vegada per cadascun dels passos en els quals hàgim desenrotllat la xarxa. Quant contribuirà exactament cada paràmetre a cada moment dependrà del pas concret del qual es tracti i dels valors d'entrada de la xarxa.

En una xarxa neuronal en la qual hi hagi connexions recurrents que vagin directament de capes posteriors a capes anteriors es pot demostrar, fent el càlcul analític, que el gradient de la funció de cost respecte als paràmetres en un pas concret de la xarxa depèn del producte del gradient en els passos posteriors. Això implica que, si calculem quant contribueix al gradient un paràmetre concret en un dels passos inicials de la xarxa, el gradient apareixerà elevat a una potència, que pot ser molt alta si hem desenrotllat la xarxa en

un gran nombre de passos intentant capturar dependències el més llunyanes possibles.

Figura 9. Flux del gradient de la funció de cost en una xarxa recurrent



El fet que el gradient es vegi multiplicat amb si mateix en el càlcul de com afecten els paràmetres en cada pas provoca dos problemes diferents però relacionats, que ja hem comentat anteriorment, però que tornem a resumir a continuació.

- **Desaparició del gradient:** si la norma del gradient és menor que 1, en multiplicar el gradient amb si mateix moltes vegades el resultat és pràcticament 0, per la qual cosa els primers passos de la xarxa escassament aporten res al càlcul del gradient. Això implica que la xarxa no serà capaç d'aprendre dependències entre valors allunyats d'una seqüència, ja que el valor dels paràmetres de la xarxa es veurà influenciat únicament pels últims passos de la recurrència.
- **Explosió del gradient:** si la norma del gradient és major que 1, en multiplicar el gradient amb si mateix moltes vegades el resultat és immensament gran. Això provoca que l'entrenament de la xarxa sigui completament inestable, ja que les modificacions que s'apliquen als paràmetres utilitzant el mètode de retropropagació fan que hi hagi massa fluctuacions i la xarxa no aconsegueix aprendre res.

2. Tipologia de cel·les recurrents

La solució més utilitzada per resoldre els problemes d'explosió i desaparició del gradient consisteix a utilitzar cel·les en les quals existeix una connexió entre l'estat anterior i la sortida de la cel·la a la qual no s'apliquen funcions no lineals. En aquesta mena de cel·les es controla la informació que flueix per aquesta connexió directa utilitzant un concepte similar a les portes lògiques, modelades amb neurones completament connectades.

En aquest capítol veurem les dues arquitectures de cel·les més utilitzades d'aquest tipus, l'arquitectura LSTM i l'arquitectura GRU.

2.1. *Long Short Term Memory* (LSTM)

En molts casos, les seqüències que volem analitzar amb una xarxa neuronal recurrent poden presentar dependències entre punts de la seqüència molt separats. Per exemple, si utilitzem una xarxa neuronal recurrent per analitzar un text és possible que dins del text es facin referències a conceptes que s'han esmentat centenars de paraules abans, per la qual cosa, si el nostre objectiu és que la xarxa sigui capaç d'extreure informació del text, és possible que necessitem que la xarxa neuronal tracti centenars de passos.

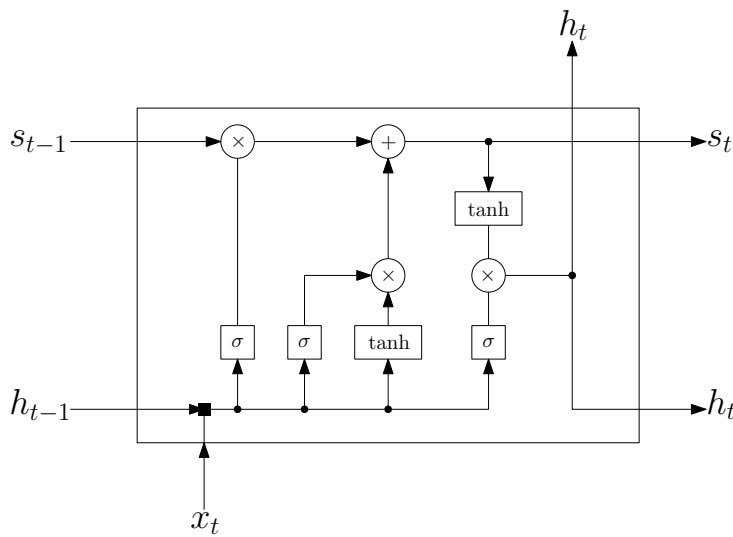
2.1.1. Estructura de les cel·les

Aquest tipus de models presenten sempre problemes com la desaparició o l'explosió del gradient. Per intentar mitigar aquests problemes, el 1997 es va proposar l'ús d'una arquitectura de cel·la molt concreta anomenada *Long Short Term Memory* (LSTM). Les entrades d'una cel·la LSTM són, d'una banda, el valor de la seqüència en el pas corresponent i, d'altra banda, la concatenació de l'estat i la sortida de la xarxa en el pas anterior. Normalment, per evitar haver de concatenar vectors a la sortida d'un pas per separar-los just a l'inici del pas següent se solen representar les cel·les LSTM amb tres entrades, com podem observar a la figura 10.

Lectura complementària

S. Hochreiter,
J. Schmidhuber. «Long
Short-Term Memory».
Neural Computation, Vol. 9
(8), p. 1735-1780, 1997.

Figura 10. Diagrama d'una cel·la LSTM



Una xarxa LSTM pot aprendre tant dependències entre punts allunyats d'una seqüència, com dependències entre punts pròxims. Els punts clau perquè la xarxa tingui aquesta habilitat són dos:

- 1) El **canal de memòria** que es pot observar com una línia horitzontal a la part superior de la cel·la.
- 2) El **control del flux d'informació** mitjançant portes.

Com podem veure al diagrama, l'arquitectura de la cel·la LSTM es basa a controlar quina informació es guarda a l'**estat** (també anomenat **memòria**) i flueix directament per la part superior i quina informació es produeix com a resposta de la xarxa. Aquest control s'exerceix mitjançant les portes següents:

Porta d'oblit

La porta d'oblit (*forget gate*) observa el valor d'entrada a la cel·la juntament amb la sortida de la xarxa en el pas anterior i decideix quina part de la memòria s'ha de conservar. Concretament, la porta d'oblit calcula un vector f_t , de la mateixa longitud que els vectors d'estat, amb la fórmula següent:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

on W_f és una matriu de paràmetres que multiplica la concatenació de h_{t-1} i x_t , i b_f és un vector de biaixos.

Per imaginar l'ús de la porta d'oblit, suposem que tenim un conjunt de dades amb el consum diari d'un cert producte i volem predir el consum durant els pròxims 3 dies. Assumim, a més, que per les característiques del producte, el

consum en un dia concret únicament depèn del consum que s'hagi produït en la mateixa setmana. En aquest cas, si en cada pas introduïm a la xarxa el consum d'un dia i afegim informació sobre el dia de la setmana del qual es tracta, és possible que la porta d'oblit aprengui a esborrar tota la memòria quan l'entrada indiqui que està tractant amb el consum d'un dilluns. D'aquesta forma, tota la informació passada serà eliminada de l'estat i els passos següents no tindran informació sobre el consum la setmana anterior, la qual cosa pot ser útil en un problema com el descrit.

Porta d'entrada

La porta d'entrada controla quina informació s'afegeix a la memòria de la xarxa. És una porta completament anàloga a la porta d'oblit, però no comparteixen paràmetres. La fórmula concreta que calcula la porta d'entrada és la següent:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

on W_i és una matriu de paràmetres i b_i és un vector de biaixos.

Com a cas d'ús simple, imaginem que estem treballant amb diferents frases i volem extreure si l'objecte principal de la frase és masculí o femení. En aquest cas, hi haurà paraules que donin informació rellevant i unes altres que no, per la qual cosa és possible que la xarxa neuronal aprengui a reconèixer les paraules que aporten informació i deixi passar únicament la informació que correspongui a les paraules útils per al problema en qüestió.

Alhora que es calculen els valors de la porta d'entrada i, utilitzant exactament la mateixa informació però amb diferents paràmetres, es calcula quina informació concretament és candidata a afegir-se a la memòria amb la fórmula següent:

$$\tilde{s}_t = \tanh(W_s \cdot [h_{t-1}, x_t] + b_s) \quad (4)$$

on W_s torna a ser una altra matriu diferent de paràmetres i b_s és un altre vector diferent de biaixos.

Finalment, una vegada s'ha calculat la informació candidata a afegir-se a la xarxa i els valors de la porta d'entrada i sortida, l'estat de la xarxa s'actualitza de la forma següent:

$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \quad (5)$$

on $\odot$ denota el producte de vectors component a component.

Com es pot apreciar a l'última fórmula, el nou estat de la xarxa ve donat per una suma ponderada entre l'estat anterior i la nova informació, on els pesos de cada component venen donats per la porta d'oblit i la porta d'entrada, respectivament.

Porta de sortida

Una vegada s'ha calculat el nou estat que tindrà la xarxa, cal decidir quina serà la sortida de la xarxa en el pas en el qual ens trobem. Per a això utilitzem la porta de sortida, el procés de la qual és similar al de la porta d'entrada. Concretament, la porta de sortida observa també el valor d'entrada a la cel·la juntament amb la sortida anterior de la xarxa i calcula un vector o_t de la forma següent:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6)$$

Aquest vector marcarà quina informació es dona com a resposta, la qual vindrà donada per la fórmula següent:

$$h_t = o_t \odot \tanh(s_t) \quad (7)$$

Com podem observar, la sortida de la xarxa consistirà en una transformació no lineal de l'estat actual de la xarxa, modulada segons el valor de la porta de sortida.

Enllaçant amb els exemples anteriors, és intuïtiu pensar que la memòria de la xarxa contindrà informació útil per donar una resposta en el pas concret en el qual es troba la xarxa, però també contindrà informació útil per als passos següents, per la qual cosa la porta de sortida s'encarrega de deixar passar únicament la informació que resulta útil en el moment actual.

2.1.2. Consideracions sobre les cel·les LSTM

Observem que totes les fórmules anteriors descriuen exactament diferents capes d'una xarxa neuronal, ja sigui amb la funció sigmoide o amb la tangent hiperbòlica com a funcions d'activació. Aquesta manera de construir les cel·les és el que dona el nom de xarxes neuronals recurrents.

El punt més interessant de controlar una memòria amb portes, com en el cas d'una LSTM, és que el control es fa de manera difusa, ja que les portes poden prendre qualsevol valor real entre 0 i 1, de manera que poden deixar passar informació només en part.

Per exemple, en el cas d'intentar predir el consum d'un producte, imaginem que la dependència no és únicament amb els dies de la mateixa setmana, però sí que sabem que la tendència en el consum disminueix un 20% els diumenges. En aquest cas concret, la xarxa neuronal podria aprendre a mantenir el valor de la tendència en alguna component del vector d'estat i , d'aquesta forma, quan l'entrada a una cel·la indiqui que s'està considerant un diumenge, és possible que la porta d'oblit multipliqui per 0.8 la component concreta de la memòria que guarda la tendència del consum.

De manera semblant, a l'exemple en el qual volem extreure informació d'una frase, és possible que no hi hagi una única paraula que contingui tota la informació, si no que per aconseguir l'objectiu del model es necessiti informació de diverses paraules. En aquest cas, la porta d'entrada de la xarxa podria aprendre a detectar quanta informació rellevant conté cada paraula i modular el pas d'informació cap a la memòria de la xarxa.

Anteriorment vam veure que les xarxes neuronals recurrents simples no són capaces de modelar llargues dependències per culpa dels problemes de desaparició i explosió del gradient. En el cas de les LSTM es pot comprovar que, gràcies a les fórmules utilitzades per calcular l'estat, la informació del gradient de la funció de cost pot fluir cap endarrere sense multiplicar-se amb si mateixa un nombre elevat de vegades. Aquest fet permet que valors d'entrada en els primers passos puguin tenir un efecte important en l'entrenament de la xarxa i aprendre així dependències entre punts allunyats d'una seqüència.

Una altra tècnica molt utilitzada que ajuda a evitar el problema de l'explosió del gradient és la retallada del gradient (en anglès, *gradient clipping*). La **retallada del gradient** consisteix a fixar un valor màxim per a cada component del gradient, de manera que si alguna component del gradient és superior al valor fixat se li assigna directament el valor màxim.

2.2. Gated Recurrent Unit (GRU)

Donat l'alt rendiment de les xarxes LSTM s'han investigat diferents variants de cel·les en les quals es controla el flux d'informació utilitzant capes de neurones com a portes que deixen passar més o menys informació cap a la memòria de la xarxa. Una de les variants que s'ha demostrat més útil es va introduir el 2014 amb el nom de *Gated Recurrent Unit* (GRU).

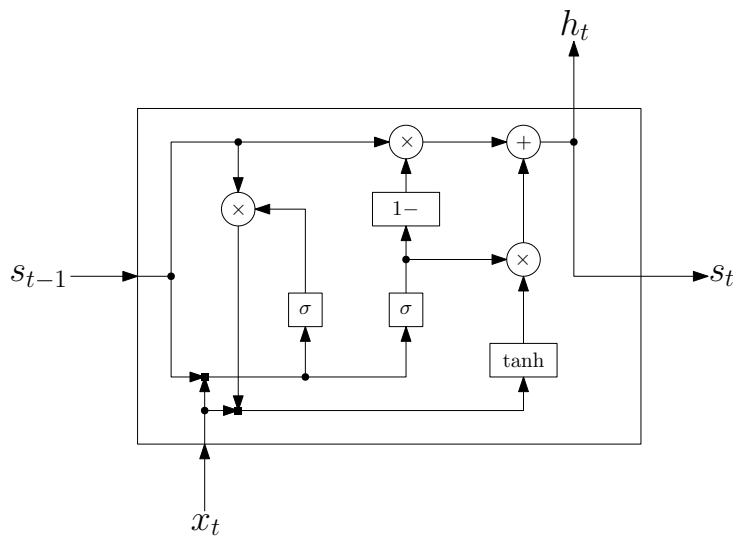
2.2.1. Estructura de les cel·les

A la figura 11 podem veure el diagrama d'una cel·la GRU, en la qual podem observar que les dues sortides que normalment hi ha en una cel·la són exactament iguals.

Lectura complementària

K. Cho *et al.* «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation». Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), p. 1724-1734, 2014.

Figura 11. Diagrama d'una cel·la GRU



Una cel·la GRU es pot veure com una simplificació d'una cel·la LSTM, en la qual es fusiona l'estat amb la resposta de la xarxa i, a més, es força al fet que l'estat de la xarxa sigui una mitjana ponderada entre l'estat anterior i la nova informació, però també existeixen lleugeres diferències en el càlcul concret que es realitza per obtenir la resposta de la xarxa.

Les xarxes recurrents amb cel·les GRU han demostrat en diversos casos donar rendiments molt similars a les xarxes recurrents amb cel·les LSTM. Per aquesta raó, les cel·les GRU són molt utilitzades quan el conjunt de dades no és extremadament gran, ja que en una cel·la GRU hi ha menys paràmetres i per tant es necessiten menys dades per a l'entrenament.

Concretament, en una cel·la GRU el flux d'informació es controla amb les portes de *reset* i les portes d'actualització, que passem a descriure a continuació.

Porta de *reset*

La porta de *reset* permet seleccionar quina informació de la memòria serà utilitzada en un pas concret. Per a això, observa l'estat anterior de la xarxa i les dades d'entrada i obté un vector que serà multiplicat posteriorment amb l'estat de la memòria. En aquest cas concret la fórmula utilitzada és la següent:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (8)$$

on W_r és una matriu de paràmetres que multiplica a la concatenació d' h_{t-1} i x_t , i b_r és un vector de biaixos.

Porta d'actualització

La porta d'actualització és anàloga a les portes d'oblit i entrada d'una cel·la LSTM. En particular, aquesta cel·la controla quina informació es manté a la memòria i quina informació de la memòria s'oblida. En primer lloc es calcula un vector de la mateixa longitud que el vector d'estat amb la fórmula següent:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (9)$$

on W_z és una matriu de paràmetres i b_z és un vector de biaixos.

A continuació es calcula la informació candidata a afegir-se a la memòria (recordem que en el cas d'una cel·la GRU el vector d'estat de la xarxa i el vector de sortida coincideixen). En una cel·la GRU aquest càlcul utilitza les dades d'entrada, l'estat anterior de la xarxa i la sortida de la porta de *reset*, tal com mostra la fórmula següent:

$$\tilde{s}_t = \tanh(W_s \cdot [r_t \odot h_{t-1}, x_t] + b_s) \quad (10)$$

on W_s és una matriu de paràmetres i b_s és un vector de biaixos.

Finalment, s'actualitza l'estat de la xarxa utilitzant una mitjana ponderada entre l'estat anterior i la informació recentment calculada:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{s}_t \quad (11)$$

2.2.2. Consideracions sobre les cel·les GRU

Com hem comentat anteriorment, una cel·la GRU és, en certa manera, una simplificació d'una cel·la LSTM, però així i tot existeixen altres variants de cel·les que simplifiquen encara més l'arquitectura de la cel·la per necessitar el mínim nombre de paràmetres possible i poder així entrenar la xarxa recurrent amb un conjunt de dades relativament petit.

3. Arquitectures de xarxes recurrents

Una vegada sabem quin tipus de cel·la volem utilitzar necessitem decidir com s'organitzaran les cel·les. En aquest capítol veurem les tècniques més comunament utilitzades per connectar cel·les recurrents de manera que puguem obtenir el màxim rendiment depenent del problema que vulguem resoldre.

3.1. Xarxes neuronals recurrents bidireccionals

En molts casos d'ús d'una xarxa recurrent la seqüència de dades té una clara component temporal, com per exemple en les sèries temporals. En altres casos d'ús, no obstant això, les dades d'entrenament estan formades per seqüències que no tenen una component temporal subjacent, com per exemple en el cas dels textos.

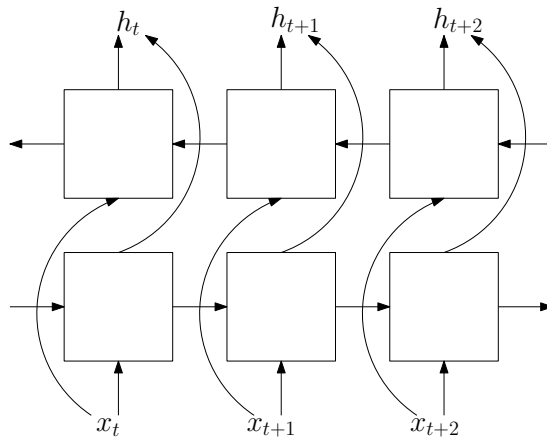
En aquests casos en els quals treballem amb seqüències aïllades sense component temporal és possible introduir els valors d'entrada a la xarxa, tant en l'ordre original de les dades com en l'ordre invers. Un exemple clar que és important l'ordre en el qual s'introdueixen les dades a la xarxa es dona quan es treballa amb textos. En efecte, depenent de l'idioma d'un text, les paraules més rellevants poden estar al principi o al final de la frase. De la mateixa forma, les construccions en les quals certes paraules es veuen modificades per paraules adjacents poden tenir diferent ordre en diferents idiomes.

En general, en aquests casos sense component temporal subjacent, per obtenir el màxim possible d'informació és possible utilitzar **xarxes neuronals recurrents bidireccionals**. En aquesta mena de xarxes primer s'aplica una recurrència en un ordre i després s'aplica una altra recurrència en l'ordre invers, per després aglutinar tota la informació recollida en una capa final. La figura 12 mostra un diagrama d'una xarxa neuronal recurrent bidireccional, en la qual es pot utilitzar una capa completament connectada per aglutinar la informació sobre els estats de les cel·les recurrents.

Lectura complementària

M. Schuster, K. K. Paliwal.
«Bidirectional Recurrent
Neural Networks». Trans.
Sig. Proc., Vol. 45 (11),
p. 2673-2681, 1997.

Figura 12. Diagrama d'una xarxa neuronal recurrent bidireccional



És important observar a la figura 12 quines són les connexions entre les diferents parts de l'arquitectura de la xarxa. En particular, podem veure que les dues recurrències de la xarxa són independents, rebent cada part de la recurrència l'estat anterior respectiu i els valors d'entrada (en ordre diferent). La informació que proporciona cada cel·la en cadascun dels passos es concatena una vegada finalitzada l'anàlisi de la seqüència, tant en un ordre com en ordre invers, i s'utilitza una capa completament connectada per extreure la informació útil segons el problema. A la secció 3.4. veurem una alternativa a la concatenació directa de totes les sortides de les cel·les.

En una xarxa neuronal recurrent bidireccional les cel·les que van en una mateixa direcció comparteixen paràmetres, però els paràmetres de les cel·les que van cap a una direcció són diferents dels paràmetres de les cel·les que van en direcció contrària. D'aquesta forma, el nombre de paràmetres en una xarxa neuronal recurrent bidireccional és el doble del que seria en una xarxa neuronal equivalent que no fos bidireccional.

Aquest tipus d'arquitectures són molt útils, per exemple, en problemes de classificació de text. En aquest cas podem entendre la xarxa neuronal com dos agents que extreuen el màxim possible d'informació del text, un llegint el text en l'ordre original i l'altre llegint el text en l'ordre invers, per després compartir la informació i fer una classificació definitiva utilitzant una capa completament connectada.

3.2. Xarxes neuronals recurrents profundes

Si observem l'arquitectura d'una cel·la LSTM o una cel·la GRU, a les figures 10 i 11, podem veure que les operacions que es realitzen sobre les dades d'entrada són com si tinguéssim una xarxa neuronal amb una única capa. És cert que existeix un cert grau de composició entre les operacions donada la recurrència de la xarxa, però aquesta composició es fa sempre utilitzant els mateixos

paràmetres i canviant únicament les dades d'entrada en cada pas. Això, com en el cas de les xarxes neuronals completament connectades, provoca que la xarxa no tingui una gran capacitat d'abstracció.

Per intentar dotar de més flexibilitat a una xarxa recurrent podem procedir com en el cas de les xarxes completament connectades i apilar capes de cel·les recurrents unes damunt d'unes altres. A la figura 13 podem veure exactament com podem connectar les entrades i les sortides de diverses cel·les per crear una **xarxa neuronal recurrent profunda**.

Figura 13. Exemple d'una xarxa neuronal recurrent profunda desenrotllada

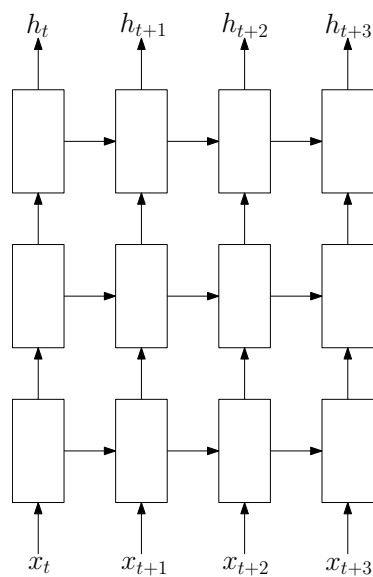
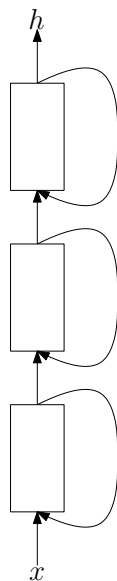


Figura 14. Xarxa neuronal recurrent de la figura 13 sense desenrotllar



És important tenir en compte quins paràmetres són compartits entre les cel·les de la xarxa. A la figura 14 podem veure la mateixa xarxa recurrent que apareix desenrotllada a la figura 13. Com podem veure, les cel·les recurrents d'una

Lectura complementària

A. Greus *et al.* «Speech recognition with deep recurrent neural networks». In International Conference on Acoustics, Speech and Signal Processing, p. 6645-6649, 2013.

mateixa capa comparteixen paràmetres, però cadascuna de les capes és independent, per la qual cosa no comparteixen paràmetres.

Si observem de nou la figura 13, podem apreciar que, a diferència del que ocorre en una xarxa recurrent com la de la figura 4, la informació que entra a la xarxa passa per diferents capes amb diferents paràmetres. Això permet a la xarxa especialitzar cada capa en conceptes lleugerament més complexos que els que tracta la capa anterior, per la qual cosa, de la mateixa forma que passa amb les CNN, la xarxa és capaç d'aprendre conceptes molt abstractes. Però, com en totes les xarxes neuronals, aquest augment de la capacitat d'abstracció ve associat a una major complexitat del procés d'entrenament.

En una xarxa neuronal recurrent profunda s'uneixen el problema de la desaparició i l'explosió del gradient a la complexitat d'entrenar una xarxa neuronal amb diverses capes. És completament indispensable disposar d'un conjunt de dades molt gran per poder entrenar una xarxa neuronal recurrent profunda.

A més, és molt important ajustar correctament els diferents hiperparàmetres de l'entrenament per obtenir una xarxa neuronal estable. Això pot provocar que hàgim de provar diferents combinacions d'hiperparàmetres buscant l'estabilitat, la qual cosa, juntament amb el temps d'entrenament de la xarxa, pot allargar molt el temps de desenvolupament d'una xarxa d'aquest tipus.

3.3. Arquitectura codificador-descodificador

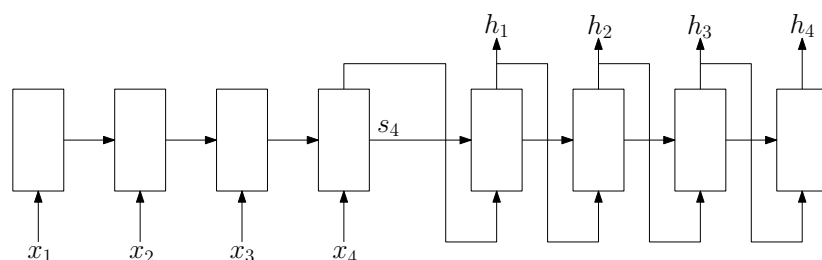
A la figura 7 hem vist que en una xarxa neuronal recurrent podem tenir, alhora, seqüències d'entrada i seqüències de sortida. Un cas particular d'aquesta combinació d'entrades i sortides el podem veure a la figura 15, en la qual primer la xarxa neuronal consumeix completament la seqüència d'entrada i, posteriorment, genera la seqüència de resposta.

En una xarxa neuronal amb aquesta mena d'arquitectura es denomina **codificador** a la primera part de la xarxa i **descodificador** a la segona part.

Lectura complementària

I. Sutskever *et al.* «Sequence to Sequence Learning with Neural Networks». Proceedings of the 27th International Conference on Neural Information Processing Systems, Vol. 2, p. 3104-3112, 2014.

Figura 15. Xarxa neuronal recurrent amb arquitectura *encoder-decoder* / *sequence-to-sequence*



La informació que passa del codificador al descodificador està tota comprimida en el **vector d'estat** de l'últim pas del codificador. En aquest sentit, el treball del codificador consisteix a guardar tota la informació de la seqüència d'entrada que sigui rellevant per al problema que estem tractant en un vector que després el descodificador sigui capaç de tractar.

D'altra banda, el treball del descodificador consisteix a crear una seqüència de sortida basada únicament en la informació que rep del codificador.

El cas d'ús típic d'aquesta mena d'arquitectura es dona en els problemes de traducció automàtica. En aquests casos, la xarxa primer consumeix tota la seqüència d'entrada per obtenir tota la informació d'una frase i després genera la frase traduïda paraula a paraula. El treball del codificador en aquest cas consisteix a comprimir tota la informació continguda en una frase en un vector, de manera que després el descodificador sigui capaç d'interpretar el contingut del vector i generar una frase en un nou idioma que tingui el mateix significat que la frase original.

Com en molts altres casos, el punt clau d'aquesta mena de xarxes neuronals està a aconseguir representar la informació rellevant en un vector numèric. Per obtenir uns vectors el més representatius possible es poden combinar les tècniques que hem vist a les seccions anteriors sobre xarxes bidireccionals i xarxes recurrents profundes.

En aquest sentit, depenent del problema, és possible que una xarxa neuronal bidireccional sigui capaç d'obtenir una millor representació vectorial de la seqüència d'entrada, de manera que tenir un codificador bidireccional farà que el descodificador pugui obtenir millors resultats.

De la mateixa forma, si les seqüències d'entrada contenen conceptes abstractes, és molt possible que utilitzant diverses capes en el codificador i en el descodificador permeti obtenir millors resultats. Aquestes millores comporten, com és d'esperar, una major complexitat en l'entrenament de la xarxa neuronal, per la qual cosa normalment és necessari disposar d'una gran quantitat de dades per entrenar una xarxa neuronal amb arquitectura codificador-descodificador.

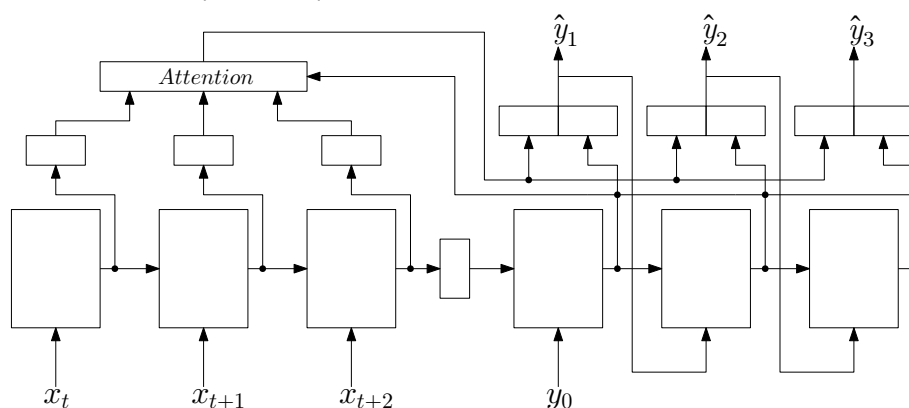
3.4. Mecanisme d'atenció

Com hem vist anteriorment, amb una arquitectura codificador-decodificador es pot entrenar un model que permeti traduir frases entre idiomes. En aquesta mena d'arquitectures, és intuïtiu pensar que la part més complexa del model està a sintetitzar en un únic vector tota la informació continguda en una frase, per després poder generar una frase en un altre idioma.

Per millorar en aquest procés de compressió de la informació, el 2015 s'introdueix el mecanisme d'atenció, l'objectiu del qual és ajudar a construir un vector que condensi la informació continguda en una frase de la millor forma possible.

Concretament, amb el **mecanisme d'atenció**, en lloc d'utilitzar el vector d'estat de l'últim pas de la xarxa recurrent com a representació de tota la seqüència, s'utilitza una suma ponderada dels estats de cada pas de la xarxa codificadora. A la figura 16 podem veure una representació del mecanisme d'atenció.

Figura 16. Mecanisme d'atenció en una xarxa neuronal recurrent amb arquitectura *encoder-decoder* / *sequence-to-sequence*



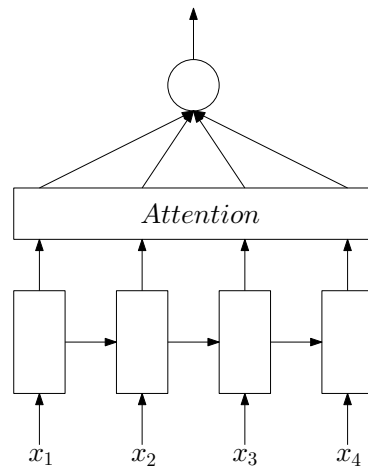
La idea fonamental és que, a la suma ponderada, els pesos que s'associen a cada pas de l'entrada depenen del pas concret de la sortida en el qual es trobi el procés de descodificació. Aquesta suma ponderada de tots els estats es concatena després amb l'estat de la xarxa descodificadora per utilitzar-se com a base en el càlcul de la sortida de la xarxa, la qual cosa permet a la xarxa centrar l'atenció en certs punts de la seqüència a cada moment. D'aquesta forma, és possible representar més fidedignament les parts importants de la seqüència en el vector que s'utilitza en cada pas del descodificador.

El mecanisme d'atenció també pot ser utilitzat amb una xarxa recurrent en un problema de regressió o classificació habitual. El procés és exactament el mateix que amb una arquitectura codificador-descodificador, però en lloc de calcular una suma ponderada dels estats del codificador per cada pas del descodificador, es realitza una única suma ponderada en finalitzar de tractar cada seqüència d'entrada. A la figura 17 podem veure un diagrama amb una xarxa neuronal amb mecanisme d'atenció aplicat a un problema de classificació.

Lectura complementària

D. Bahdanau *et al.* «Neural Machine Translation by Jointly Learning to Align and Translate». 3rd International Conference on Learning Representations, 2015.

Figura 17. Mecanisme d'atenció en una xarxa neuronal recurrent per un problema de classificació



Aquest mecanisme pot ser molt útil en casos en els quals una part d'una seqüència és la que porta tota la informació rellevant per al problema. Per exemple, si volem extreure el temps verbal d'una frase és probable que la informació rellevant es trobi en una zona molt concreta de la frase que no té per què estar al final. Utilitzant un mecanisme d'atenció, la xarxa pot aprendre a detectar els verbs, aïllar-los i després classificar el temps verbal sense parar esment a altres parts de la frase que poden aportar soroll.

4. Consells pràctics i exemples

En aquest últim capítol del mòdul de xarxes neuronals recurrents ens centrarem en veure alguns consells pràctics a l'hora de treballar amb RNN i sèries temporals per a la creació de models predictius. Finalment, veurem alguns exemples de problemes que es poden resoldre emprant les RNN.

4.1. Consells pràctics en l'ús d'RNN

Durant aquesta part dedicada a les RNN hem vist alguns exemples de problemes que es poden resoldre amb xarxes neuronals recurrents. No obstant això, és important tenir present que cada problema té unes característiques especials que requeriran diferents solucions, ja sigui en l'arquitectura de la xarxa neuronal o en el processament de les dades.

En el cas de treballar amb sèries temporals, per exemple, és important utilitzar tècniques estadístiques clàssiques per preprocessar les dades, com la normalització o l'eliminació de tendències, per obtenir els millors resultats possibles de l'entrenament.

Una de les tècniques més utilitzades és la descomposició d'una sèrie temporal en tres components: tendència, estacionalitat i component irregular. Aquesta descomposició es pot realitzar de dues formes diferents, additiva i multiplicadora. Si tenim una sèrie temporal y_t , aplicant una descomposició additiva obtenim

$$y_t = T_t + S_t + I_t$$

mentre que si apliquem una descomposició multiplicadora obtenim

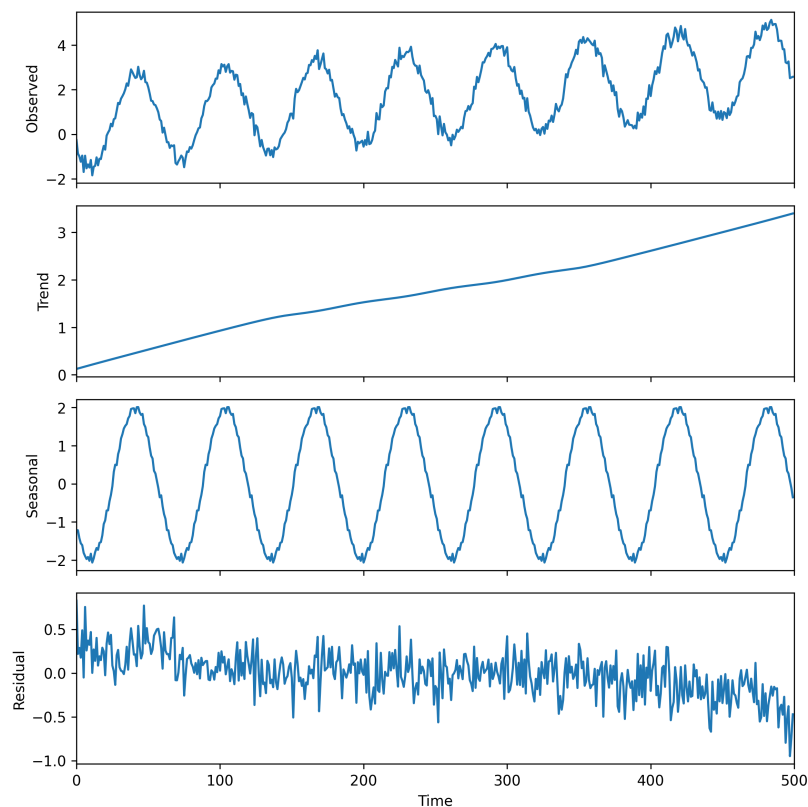
$$y_t = T_t \cdot S_t \cdot I_t$$

on T_t denota la tendència, S_t l'estacionalitat i I_t la component irregular. A la figura 18 podem veure un exemple d'una sèrie temporal descomposta de manera additiva.

Utilitzant la descomposició en tres components transformem el problema d'entrenar una xarxa neuronal per predir una sèrie temporal a haver de predir tres sèries temporals diferents. L'avantatge en aquest cas és que les compo-

nents seran més fàcils de predir, especialment la part de la tendència i de l'estacionalitat.

Figura 18. Sèrie temporal i la seva descomposició en tendència, estacionalitat i component irregular



Posteriorment, quan vulguem predir un nou valor de la sèrie original, simplement haurem de predir els valors de les tres components i agregar-los utilitzant el mateix mètode que utilitzem per la descomposició, ja sigui additiva o multiplicadora.

En el cas que estiguem utilitzant una xarxa neuronal recurrent per treballar amb textos, és molt important seleccionar una forma adequada per representar numèricament les paraules que formen cada registre.

La forma més simple de representar un text consisteix a fixar primer un diccionari amb N paraules (per exemple, 1.000, 2.000 o 5.000 paraules). Llavors, per cada text es construeix un vector de N dimensions per a cadascuna de les paraules. Cada vector té totes les components igual a zero excepte una component, que té el valor 1, i indica quina paraula del diccionari estem representant amb el vector.

Aquesta manera de representar textos té diversos inconvenients, com per exemple:

- Cada paraula està descrita per un vector d' N dimensions, on N pot ser un valor molt gran si el diccionari conté moltes paraules. Per tant, no és adequat, en la majoria dels casos, perquè incrementa de manera considerable la grandària de la xarxa neuronal, augmentant la complexitat de l'entrenament.
- Les paraules tenen relacions de similitud que no es veuen reflectides en els vectors. És a dir, els vectors corresponents a dues paraules que tinguin significat semblant tindran la mateixa relació que els vectors corresponents a dues paraules completament diferents.

Això torna a incrementar la complexitat de l'entrenament, ja que dos textos que tinguin significats similars tindran representacions vectorials sense cap relació, per la qual cosa la xarxa haurà d'aprendre per separat sobre els dos textos i no podrà utilitzar informació apresada amb un exemple a l'hora de tractar amb exemples relacionats.

Per solucionar aquests dos problemes se sol utilitzar una capa d'*embedding* just abans de l'entrada de la xarxa neuronal.

Aquesta capa és simplement una matriu de grandària $N \times k$, on N és el nombre de paraules que tenim al diccionari i k és la dimensió de l'*embedding*, que sol estar entre 50 i 300, i.e. $k \in \{50, \dots, 300\}$.

Aquesta capa d'*embedding* assigna a cada paraula un vector amb k valors reals, per la qual cosa és possible representar les N paraules del diccionari amb només k components. Perquè aquest procés sigui útil, la representació de cada paraula amb k valors ha de complir la propietat que paraules amb significat similar tinguin valors similars, mentre que paraules amb significat diferent tinguin valors diferents. Això es pot aconseguir de dues formes principalment:

- Crear la matriu d'*embedding* de forma no supervisada com a preentrenament de la xarxa neuronal recurrent.

En aquest cas, el procés consisteix a utilitzar un algorisme com Word2Vec, que és capaç d'extreure relacions semàntiques entre paraules de forma no supervisada observant únicament els textos disponibles per al problema que intentem resoldre, sense cap altra forma de supervisió.

- Crear la matriu d'*embedding* alhora que s'entrena la xarxa neuronal recurrent.

En aquest cas es comença amb una matriu d'*embedding* amb valors aleatoris i es consideren les entrades de la matriu com a paràmetres de la xarxa neuronal durant l'entrenament.

Lectura complementària

T. Mikolov *et al.*
«Distributed Representations of Words and Phrases and Their Compositionality». Proceedings of the 26th International Conference on Neural Information Processing Systems, Vol. 2, p. 3111-3119, 2013.

El propi procés d'entrenament de la xarxa neuronal ajustarà els valors dels vectors perquè cada paraula quedi representada en k dimensions de tal forma que la resposta final de la xarxa sigui la millor possible.

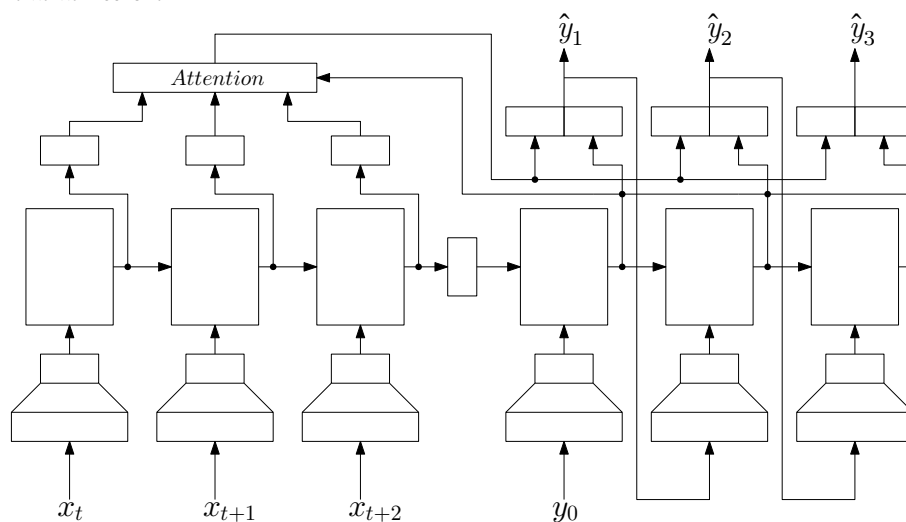
En el cas d'optar per una matriu d'*embedding* preentrenada és possible utilitzar *embeddings* disponibles a Internet que ja estan preentrenats amb grans quantitats de textos, com per exemple la Viquipèdia.

Per triar entre una forma o una altra de crear la matriu d'*embedding* haurem de tenir en compte diferents punts, com per exemple si disposem de suficients textos com per crear un *embedding* de forma no supervisada o si el vocabulari dels nostres textos és massa específic com per utilitzar un *embedding* genèric obtingut d'Internet.

També pot ser interessant barrejar les dues aproximacions i crear la matriu d'*embedding* utilitzant primer un preentrenament no supervisat i afinant després els valors dels vectors durant l'entrenament de la xarxa neuronal.

A la figura 19 podem veure el mateix esquema del mecanisme d'atenció de la figura 16 però amb la capa d'*embedding* aplicada sobre les entrades de la xarxa.

Figura 19. Mecanisme d'atenció en una xarxa neuronal recurrent amb arquitectura *encoder-decoder* / *sequence-to-sequence*, aplicant primer una capa d'*embedding* a l'entrada de la xarxa neuronal



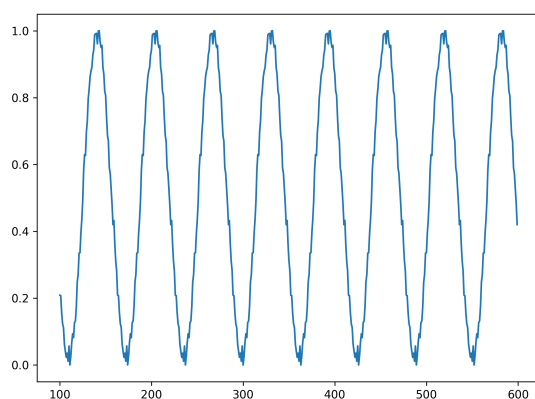
4.2. Exemples

A continuació veurem com podem predir valors futurs de la sèrie temporal de la figura 18 utilitzant la seva descomposició.

Aquesta sèrie temporal ha estat generada sintèticament, per la qual cosa sabem que tot allò que no pertany a la tendència o a l'estacionalitat és soroll blanc. En aquest sentit, l'exemple següent mostra com es podrien predir valors futurs utilitzant una xarxa neuronal recurrent a la part estacionària de la sèrie. En un cas amb dades reals es podria utilitzar un model més simple per a la part estacionària i entrenar una xarxa recurrent per intentar detectar patrons a la component irregular, però en el cas de la sèrie de la figura 18 sabem que, per construcció, sofriríem de sobreajustament (estariem entrenant un model amb un conjunt de dades format completament per soroll).

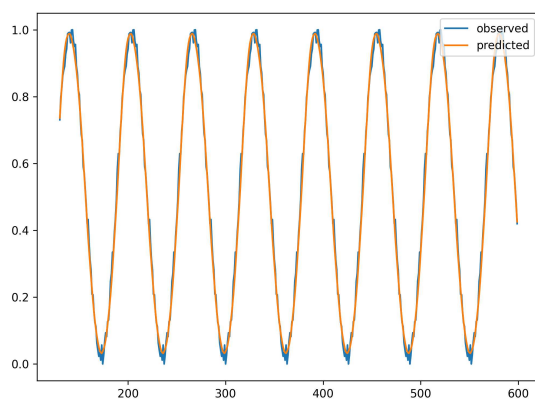
El primer pas del procés consisteix a entrenar una xarxa neuronal recurrent que capturi l'estacionalitat de les dades. Per a això, és necessari normalitzar els valors de la part estacional de la sèrie, com podem veure a la figura 20.

Figura 20. Part estacional de la sèrie normalitzada



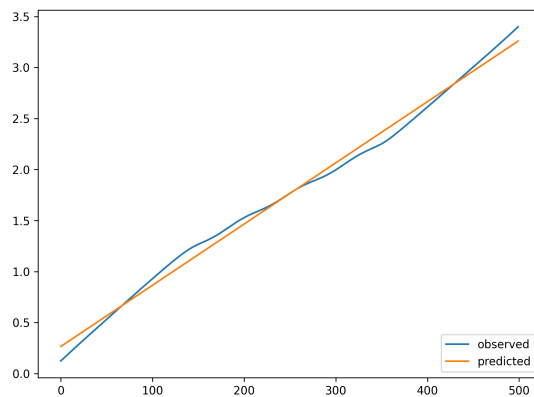
Per predir la part estacional de la sèrie podem utilitzar una xarxa LSTM amb 50 unitats en cada cel·la i que utilitzi els 30 últims valors de la sèrie per predir el següent. Amb una sèrie tan periòdica i una xarxa neuronal recurrent es poden obtenir resultats molt bons, com es mostra a la figura 21.

Figura 21. Predicció de les dades d'entrenament de la part estacional de la sèrie normalitzada



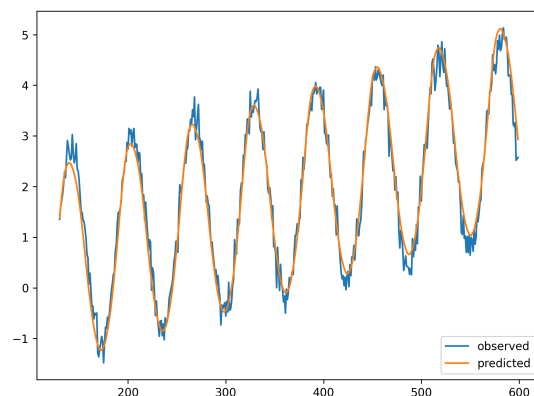
Per predir la tendència, podríem utilitzar també una xarxa neuronal, però és molt millor utilitzar un model més simple per evitar el sobreajustament. En aquest cas, atès que la tendència és pràcticament lineal utilitzarem un model lineal de regressió simple. A la figura 22 es pot veure el resultat d'ajustar la tendència de les dades d'entrenament a un model lineal.

Figura 22. Predicció de la tendència de les dades d'entrenament utilitzant un model lineal



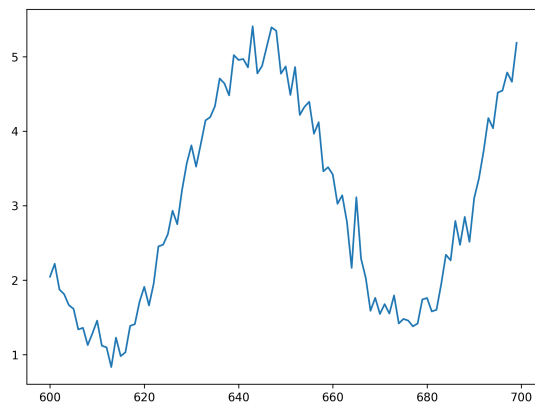
Si sumem ara les prediccions del model lineal amb les prediccions de la xarxa neuronal recurrent obtenim una predicció per les dades d'entrenament, que es pot veure a la figura 23.

Figura 23. Suma de les prediccions del model lineal i la xarxa neuronal recurrent per a les dades d'entrenament



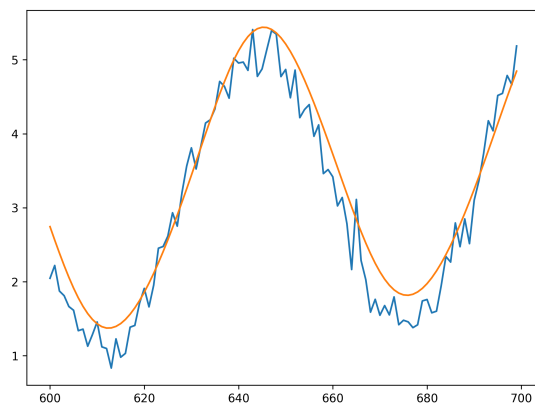
Atès que la xarxa sembla haver captat la tendència i l'estacionalitat de les dades podem utilitzar-la per a la predicció de dades futures. A la figura 24 podem veure els valors de la sèrie següents.

Figura 24. Valors de la sèrie posteriors als utilitzats per l'entrenament



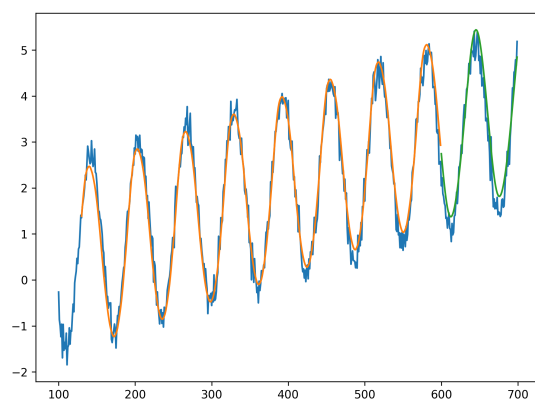
Si apliquem els models ja entrenats per predir les noves dades obtenim els resultats que mostra la figura 25.

Figura 25. Predicció dels models per a les dades de test



Finalment, podem veure totes les dades i les prediccions de manera conjunta a la figura 26.

Figura 26. Resum de prediccions per a la sèrie temporal (dades d'entrenament i de test)



En aquest cas en particular les dades de la sèrie temporal han estat generades sintèticament, per la qual cosa sabem segur que la part residual de la descom-

posició està formada principalment de valors aleatoris. Això és així en general, per la qual cosa, en principi, no s'ha d'entrenar cap model sobre la part residual, ja que sinó, per definició, sofriríem de sobreajustament.

Hi ha casos en canvi, en els quals és possible que hi hagi algun patró amagat a la part residual de la descomposició, especialment si els valors de la sèrie temporal es veuen afectats per variables externes que no s'estan considerant a l'hora de fer la descomposició. En aquests casos pot ser interessant entrenar una xarxa neuronal amb totes les variables que es considerin convenientes per intentar extreure una mica d'informació dels residus. Per exemple, si la sèrie temporal representa el consum d'energia d'un poble turístic, és possible que en la descomposició es detectin els caps de setmana o els mesos d'estiu, però hi ha festivitats com la Setmana Santa que canvien de data cada any. En aquest cas, un model en el qual s'indiqui quins dies seran festius i quins no és possible que aconsegueixi predir amb sentit part dels residus sense caure en el sobreajustament.

Resum

En aquest mòdul hem vist els conceptes bàsics de les xarxes neuronals recurrents. Hem vist que aquest tipus de xarxes és molt útil quan les dades tenen una estructura seqüencial, ja que una xarxa recurrent tractarà cada registre del conjunt de dades també de manera seqüencial.

Com hem comentat, en general, una xarxa neuronal recurrent podria tenir les connexions recurrents establertes de moltes formes diferents, però se solen utilitzar cel·les per facilitar el treball. Els tipus de cel·les més utilitzats actualment són les cel·les LSTM i les cel·les GRU.

El bon funcionament de les cel·les LSTM i GRU es basa principalment en dos factors:

- Tenen un canal de memòria al qual no se li aplica en cap moment una operació que no sigui lineal.
- Controlen el flux d'informació utilitzant funcions sigmoide per simular portes lògiques.

Depenent de la mena de problema, i especialment si es disposa de gran quantitat de dades, pot ser una bona opció combinar les cel·les d'una xarxa neuronal recurrent en arquitectures complexes. Hem vist en aquest capítol l'opció de crear xarxes neuronals recurrents bidireccionals, profundes o amb arquitectura codificador-descodificador. És possible també unir diferents tipus d'arquitectures per aconseguir xarxes neuronals amb una gran capacitat predictiva.

Finalment, hem vist els fonaments del mecanisme d'atenció, el qual és la base de l'arquitectura de transformadors que veurem en futurs mòduls.

Glossari

atenció *f* Mecanisme que permet a una xarxa neuronal amb arquitectura seq2seq utilitzar informació focalitzada de la seqüència d'entrada segons sigui l'estat de la descodificació.

cel·la *f* Model abstracte per representar l'arquitectura d'una xarxa neuronal com a combinació d'operacions amb dues entrades i dues sortides.

GRU *m Gated Recurrent Unit*, tipus de cel·la utilitzada en xarxes neuronals recurrents. Es pot considerar com una simplificació d'una cel·la LSTM.

LSTM *m Long Short Term Memory*, tipus de cel·la molt utilitzada en xarxes neuronals recurrents.

recurrència *f* Situació en la qual la sortida de les neurones d'una xarxa neuronal es connecta amb l'entrada de les neurones d'una capa anterior.

seq2seq *f* Arquitectura de xarxa neuronal amb dues parts fonamentals, un codificador i un descodificador, molt utilitzada quan el problema a tractar consisteix en la transformació de seqüències d'entrada en seqüències de sortida (per exemple, en problemes de traducció automàtica).

Bibliografia

Bahdanau, D. et al. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. 3rd International Conference on Learning Representations.

Casas Roma, J., Lozano Bagén, T., Bosch Rué, A. (2020). *Deep learning: Principios y fundamentos*. Barcelona: Editorial UOC.

Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), p. 1724-1734.

Graves, A. et al. (2013). *Speech recognition with deep recurrent neural networks*. International Conference on Acoustics, Speech and Signal Processing, p. 6645-6649.

Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, Vol. 9 (8), p. 1735-1780.

Mikolov, T. et al. (2013). *Distributed Representations of Words and Phrases and Their Compositionality*. Proceedings of the 26th International Conference on Neural Information Processing Systems, Vol. 2, p. 3111-3119.

Schuster, M., Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks*. Trans. Sig. Proc., Vol. 45 (11), p. 2673-2681.

Sutskever, I. et al. (2014). *Sequence to Sequence Learning with Neural Networks*. Proceedings of the 27th International Conference on Neural Information Processing Systems, Vol. 2, p. 3104-3112.